Debreceni SZC Mechwart András Gépipari és Informatikai Technikum

Szoftverfejlesztő és -tesztelő

Vizsgaremek

Gamehunt Játékkatalógus

Készítette: Böjtös Attila
Horváth Botond
Kuik-Filep Balázs

2026.

Gamehunt Játékkatalógus

Dokumentáció

Projekt Név
Gamehunt Játékkatalógus

Verzió
v1.0

Készítés Dátuma
2025. December - 2026. Április

Dokumentum Típusa
Technikai Dokumentáció

Fejlesztők
- Böjtös Attila: Backend
- Horváth Botond: Frontend
- Kuik-Filep Balázs: Adatbázis

GAMEHUNT

Tartalomjegyzék

TARTALOMJEGYZÉK
1. BEVEZETÉS
    1.1 A Projekt célja
    1.2 A Fejlesztői Csapat

2. RENDSZERARCHITEKTÚRA
    2.1 Frontend Architektúra
    2.2 Backend Architektúra
    2.3 Technológiai Stack

3. TELEPÍTÉSI ÚTMUTATÓ
    3.1 Előfeltételek
    3.2 Github Repository
    3.3 Render Hosztolás
    3.4 Lokális Hosztolás

4. A GAMEHUNT FELÉPÍTÉSE
    4.1 Frontend Dokumentáció
    4.2 Home Page
    4.3 Login / Register
    4.4 Favorites
    4.5 Admin View
    4.6 Backend Dokumentáció
    4.7 Render REST API
    4.8 Adatbázis Modell
    4.9 MongoDB

4.10 IGDB API
4.11 Node.js Szerver
4.12 Endpointok

5. TERVEZÉS ÉS DESIGN
5.1 UI/UX Tervezés
5.2 Színpaletta
5.3 Reszponzivitás

6. TESZTELÉS
6.1 Jest Tesztek

7. FELHASZNÁLÓI DOKUMENTÁCIÓ
7.1 Navigáció a Kezdő Oldalon
7.2 Játékok szűrése
7.3 Bejelentkezés
7.4 Regisztrálás
7.5 Kedvencek Hozzáadása
7.6 Admin Felület

8. TOVÁBBFEJLESZTÉSI LEHETŐSÉGEK
8.1 Rövid távú (1-3 hónap)
8.2 Középtávú (3-6 hónap)
8.3 Hosszú távú (6+ hónap)

9. LINKEK ÉS FORRÁSOK
9.1 Github Repository
9.2 Használt Npm Packagek

9.3 Munkamenet Nyomonkövetése

10. ÖSSZEFOGLALÁS
10.1 A Projekt Státusza
10.2 Elért Célok
10.3 Tanulságok
10.4 Elérhetőségek

1. Bevezetés

1.1 A Projekt célja

Projekt Célja: Egy platformfüggetlen, személyre szabott játékajánló webalkalmazás (a "Steam Interactive Recommender" mintájára), amely megszünteti a céltalan keresgélést, és lés (fejlesztés alatt) célzott találatokkal segíti a felhasználókat.

Főbb Jellemzők és Működés:

- **Intelligens ajánlórendszer:** Regisztráció és bejelentkezés után a rendszer folyamatosan alkalmazkodik a felhasználó ízléséhez, egyedi preferenciák alapján finomítva a találatokat.
- **Vendég mód (Guest):** Bejelentkezés nélkül is elérhető egy gyorskereső funkció, amely – adatok hiányában – az aktuálisan legnépszerűbb játékokat listázza.
- **Bővített keresőmotor:** Fejlett szűrési lehetőségek (például címkék/tagek, népszerűség és egyéb paraméterek alapján).

GAMEHUNT

- **Profilkezelés:** Teljes körű felhasználói beállítások és egyéni játékpreferenciák mentése a bejelentkezett fiókok számára.

1.2 A Fejlesztői Csapat

A projekt megvalósításáért felelős csapat az alábbi munkamegosztásban végezte el a rendszertervezési és implementációs feladatokat:

- **Böjtös Attila (Backend fejlesztő):**
  - A szerveroldali logika kidolgozása **Node.js** környezetben.
  - A **RESTful API** architektúra tervezése és teljes körű implementációja.
  - Az **IGDB API** integrációja, a külső adatforrásokkal való kommunikációs protokollok kialakítása.
- **Horváth Botond (Frontend fejlesztő):**
  - A felhasználói felület (UI) és a felhasználói élmény (UX) megvalósítása.
  - A kliensoldali programozás **JavaScript**, **HTML5** és **CSS3** technológiák alkalmazásával.
- **Kuik-Filep Balázs (Adatbázis-tervező):**
  - Az adatmodell kialakítása **MongoDB** (NoSQL) környezetben.
  - Az adatszerkezet logikai leképezése; az **ER modell** elkészítése a MongoDB kollekció-struktúrák alapján.

GAMEHUNT

2. Rendszer Architektúra

2.1 Frontend architektúra

A Gamehunt alkalmazás felhasználói felülete (UI) és a hozzá kapcsolódó felhasználói élmény (UX) a modern webes szabványok szerint, a platformfüggetlenség és a reszponzivitás jegyében készült. A kliensoldali fejlesztési feladatok megvalósításáért Horváth Botond felelt.

Technológiai Specifikációk:

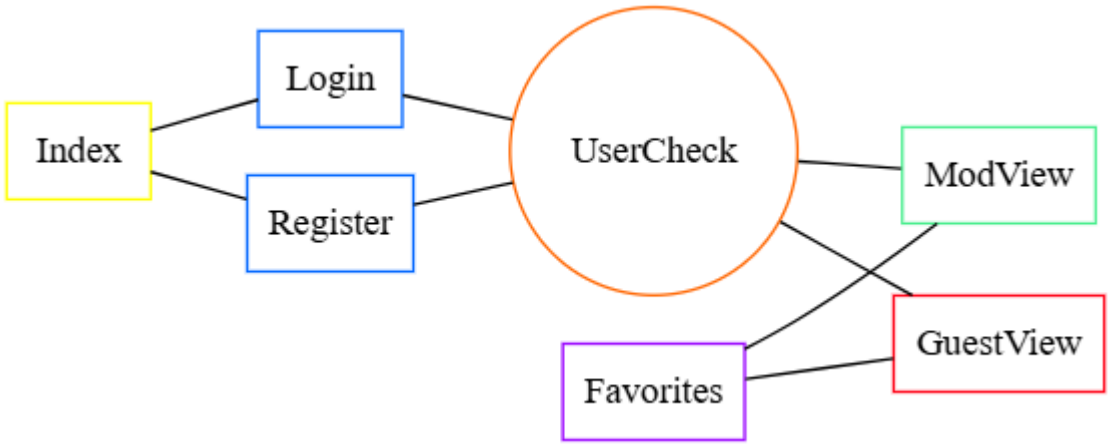
- **Alapnyelvek:** A programozás során tiszta JavaScript, HTML5 és CSS3 technológiákat alkalmaztunk.
- **Design szemlélet:** A tervezés során kiemelt figyelmet kapott a konzisztens színpaletta és a különböző eszközméretekhez való rugalmas alkalmazkodás (reszponzivitás).

Navigációs és Hozzáférési Modell:

A frontend felépítése egy központosított logikát követ, amely biztosítja a biztonságos és differenciált hozzáférést a funkciókhoz:

- **Központi vezérlés (UserCheck):** A rendszer egy belső állapotellenőrző mechanizmuson keresztül irányítja a felhasználót a megfelelő nézetekhez a bejelentkezési státusz és jogosultság alapján.
- **Belépési pontok:** Az alkalmazás az **Index** (kezdőoldal) felületen keresztül érhető el, ahonnan a felhasználó a **Login** (bejelentkezés) vagy **Register** (regisztráció) modulok felé navigálhat.
- **Nézetek (Views):**

- **GuestView:** A nem regisztrált látogatók számára biztosított vendég mód, amely az aktuálisan népszerű játékok listázására korlátozódik.
- **Favorites:** A hitelesített felhasználók egyéni profilja, ahol a mentett preferenciák és kedvenc játékok érhetőek el.
- **ModView/Admin Felület:** Dedikált felület az adminisztratív jogosultságokkal rendelkező felhasználók számára a rendszer kezeléséhez.



2.2 Backend architektúra

A Gamehunt rendszer szerveroldali logikája Node.js környezetben, Express.js keretrendszer alkalmazásával készült. A backend feladata a kliensoldali kérések kiszolgálása, az adatbázis-műveletek koordinálása, valamint a külső API-kkal való biztonságos kommunikáció fenntartása.

Alkalmazott technológiai stack:

- **Runtime környezet:** Node.js.
- **Web keretrendszer:** Express.js (RESTful API architektúra).
- **Adatbázis és ODM:** MongoDB Atlas felhőalapú NoSQL adatbázis, Mongoose objektum-adatmodell kezelővel.
- **Biztonság és titkosítás:** Bcryptjs a jelszavak egyirányú titkosításához és UUID v4 a felhasználók globálisan egyedi azonosításához.
- **Külső integráció:** IGDB (Twitch) API interakció Node-fetch modul segítségével.

Főbb funkcionális egységek:

1. **Adatkezelés és Modellezés:** A rendszer az adatokat MongoDB kollekciókban tárolja. A User modell központosítottan kezeli a felhasználói adatokat, beleértve a hitelesítési információkat, a kedvenc játékok listáját (favorites), valamint a jogosultsági szinteket (isAdmin, adminLevel).
2. **Hitelesítési Modul (Auth):**
   A rendszer biztonságos regisztrációs és bejelentkezési folyamatot implementál:
   - **Regisztráció:** Ellenőrzi az adatok egyediségét (email, username), kiszámítja a felhasználó életkorát a születési dátum alapján az isAdult státusz meghatározásához, és titkosított formában tárolja a jelszót.
   - **Bejelentkezés:** Case-insensitive felhasználónév-keresést és Bcrypt alapú jelszó-összehasonlítást végez.

GAMEHUNT

3. **IGDB API és Token Kezelés:**
   A szerver felelős a külső játék adatbázishoz való hozzáférésért. Tartalmaz egy automatizált OAuth2 mechanizmust, amely lekéri és frissíti a Twitch access_token-t a biztonságos API hívásokhoz. A lekérdezések (pl. /games/popular) komplex szűrő feltételeket tartalmaznak, például az életkor-alapú korlátozások (themes != 42) érvényesítését.
4. **Hierarchikus Adminisztrációs Rendszer:**
   A backend egy többszintű jogosultság kezelő rendszert valósít meg (requireAdmin middleware), amely 0-tól 3-ig terjedő adminisztrátori szinteket különít el:
   - **Szint alapú korlátozás:** Az adminisztrátorok csak a náluk alacsonyabb vagy velük egyenlő szintű felhasználókat módosíthatják (bizonyos műveletek esetén szigorúbb korlátozásokkal).
   - **Műveletek:** Felhasználók törlése, átnevezése (kizárólag Level 0), admin státusz és életkor-státusz módosítása.
5. **Hibakezelés és Tesztelhetőség:**
   A szerver fel van készítve a MongoDB kapcsolat automatikus újrapróbálkozására (retry logic), valamint SIGINT/SIGTERM szignálok kezelésével biztosítja az adatbázis-kapcsolatok tiszta lezárását. A moduláris felépítés lehetővé teszi a Jest keretrendszerrel történő automatizált tesztelést.

2.3 Technológiai Stack

A projekt a szoftverfejlesztés iparági standardjait

GAMEHUNT

követve három fő rétegre tagolódik: kliens oldal (Frontend), szerver oldal (Backend) és adatkezelés (Database).

Réteg	Technológia / Könyvtár	Funkció
Front end	HTML5, CSS3, JavaScript	Felhasználói felület és interakciók.
Backe nd	Node.js, Express.js	Szerver Oldali logika és REST API.
Adatb ázis	MongoDB Atlas, Mongoose	NoSQL adatmodell és felhő alapú tárolás.
Külső API	IGDB (Twitch) API	Játék Adatok és metaadatok forrása.
Bizto nság	Bcryptjs, UUID v4	Jelszó-hashelés és egyedi azonosítók.
Teszte lés	Jest	Automatikus egység- és integrációs tesztek.

GAMEHUNT

Hosztolás	Render, GitHub	Verziókezelés és felhőalapú futtatás.

3. Telepítési Útmutató

3.1 Előfeltételek

A szoftver futtatásához a következő szoftveres környezet és hozzáférések megléte elengedhetetlen:

- **Runtime környezet:** Node.js (v16.x vagy újabb javasolt) és npm csomagkezelő.
- **Adatbázis:** Aktív MongoDB Atlas klaszter és a hozzátartozó kapcsolati karakterlánc (Connection String).
- **API hozzáférés:** Twitch Developer regisztráció az IGDB adatok lekéréséhez szükséges CLIENT_ID és CLIENT_SECRET kulcsok generálásához.
- **Környezeti változók:** Egy .env fájl konfigurálása a fenti titkos kulcsok tárolására.

3.2 Github Repository

A forráskód verziókezelése a GitHub platformon történik. A projektet a következő lépésekkel lehet lehúzni:

1. Nyisson egy terminált a kívánt munka könyvtárban.

GAMEHUNT

2. Adja ki a git clone https://github.com/23BAttila/Vizsgaremek/ parancsot (a pontos URL a dokumentáció 10.1-es fejezetében található).
3. Lépjen be a projekt gyökérkönyvtárába.

3.3 Render Hosztolás

A Gamehunt alkalmazás felhőalapú futtatása a Render platformon valósítható meg, amely közvetlen integrációt biztosít a GitHub tárhelyhez:

1. **Web Service létrehozása:** Kapcsolja össze Render fiókját a GitHub repository-val.
2. **Környezeti változók beállítása (Environment Variables):** A "Settings" menüpont alatt kötelezően rögzíteni kell a MONGO_URI, CLIENT_ID és CLIENT_SECRET változókat.
3. **Build & Start Command:**
   - Build Command: npm install
   - Start Command: node server.js
4. **A sikeres telepítést követően** az alkalmazás a Render által generált publikus URL-en keresztül elérhetővé válik.

3.4 Lokális Hosztolás

Helyi fejlesztői környezetben az oldal az alábbi módon indítható el:

1. **Függőségek telepítése:** Adja ki az npm install parancsot a szükséges modulok letöltéséhez.
2. **Konfiguráció:** Hozzon létre egy .env fájlt a gyökérkönyvtárban, és másolja bele a szükséges kulcsokat.

3. **Szerver indítása:** Futtassa a node server.js parancsot a terminálban.
4. **Elérés:** A szerver alapértelmezetten a http://localhost:3000 címen hosztolja az oldalt, ahol a public mappában található statikus állományok automatikusan kiszolgálásra kerülnek.

4. Gamehunt Felépítése

A Gamehunt alkalmazás szerkezete a modern, aszinkron webfejlesztési elvekre épül, éles határvonalat húzva a kliensoldali megjelenítés és a szerveroldali adatfeldolgozás közé.

4.1 Frontend Dokumentáció

A kliensoldali felület egy egyoldalas alkalmazás (SPA) működési elvét szimulálja, minimalizálva az oldal újratöltését.

- Struktúra: A vizuális vázat a szemantikus HTML5 elemek és a CSS3 alapú, reszponzív rácsszerkezet (grid layer) biztosítja.
- Modularitás: A JavaScript logika funkcionális modulokra van bontva (pl. game_details.js, user.js, admin.js, gameapi.js), amelyeket az index.html a dokumentum végén tölt be a renderelési blokkolás elkerülése érdekében.

4.2 Home Page

A kezdőoldal (Homepage) az alkalmazás központi navigációs hubja.

- Navigációs sáv (Sidebar): Gyors hozzáférést biztosít a Főoldalhoz, Keresőhöz, Kategóriákhoz (Category Drawer) és a Kedvencekhez.
- Dinamikus tartalom: A játékok adatai aszinkron módon töltődnek be. A felhasználó egy játékra kattintva egy modális ablakban (Game Details Modal) tekintheti meg a részleteket, ahol a game_details.js felel az adatok (címkék, platformok, megjelenési dátumok) formázásáért és a nagy felbontású borítóképek betöltéséért.
- Interakció: Az oldal támogatja a billentyűzet-alapú navigációt, például a képernyőképek közötti lapozást (nyílbillentyűk) és a modális ablakok bezárását (Escape).

4.3 Login / Register

A hitelesítési modul biztosítja a felhasználói munkamenetek (session) kezelését kliens oldalon.

- Felület: A bejelentkezés és regisztráció dedikált modális ablakokban történik (login-modal, register-form-container).
- Munkamenet kezelés: Sikeres azonosítás után a rendszer a localStorage API-t használja a felhasználó állapotának (pl. currentUser, isAdult) perzisztens tárolására.
- Visszacsatolás: A műveletek eredményéről (sikeres belépés, hibás jelszó) a rendszer vizuális értesítéseket (Toast) küld a felhasználónak.

4.4 Favorites

A személyre szabott tartalomkezelés alapja a kedvencek funkció.

- Jogosultság-ellenőrzés: A user.js a hivatkozásra (Favorites link) történő kattintáskor validálja, hogy létezik-e aktív munkamenet (currentUser). Ha nem, a rendszer megakadályozza a navigációt és automatikusan megnyitja a bejelentkezési ablakot.
- Adatlekérés: Hitelesített felhasználó esetén a kliens egy GET kérést indít a /api/favorites/${currentUser} végpontra a mentett játékok lekéréséhez.

4.5 Admin View

Az adminisztratív funkciók egy dinamikusan generált, védett felületen érhetők el.

- Hozzáférhetőség: A jogosultság ellenőrzése a /api/admin/level/${currentUser} végponton történik; az Admin gomb csak megfelelő jogosultsági szint (adminLevel) esetén kerül beillesztésre a DOM-ba.
- Interfész: A felület egy átméretezhető modális ablak (admin-modal-content), amely egy szűrhető adatbázis-táblázatot jelenít meg (felhasználónév és e-mail alapú szűrés).
- Műveletek: Az adminisztrátorok a kliensből indíthatnak állapot módosító hívásokat (felhasználó törlése, felnőtt tartalom kapcsolása, jogosultsági szint módosítása) a megfelelő API végpontok felé.

GAMEHUNT

4.6 Backend Dokumentáció

A szerver oldal felel a teljes üzleti logikáért és az adatok integritásáért.

- Technológia: Node.js futtató környezetre épülő Express.js webkiszolgáló.
- Biztonság: A külső eléréseket a CORS middleware szabályozza, a szenzitív konfigurációs adatok (API kulcsok, DB URI) pedig környezeti változókból (dotenv) töltődnek be. A statikus fájlok (HTML, CSS, JS) kiszolgálása a public mappából történik.

4.7 Render REST API

A backend szabványos RESTful architektúrát valósít meg a kliens-szerver kommunikációhoz.

- Adatcsere formátuma: A kérések és válaszok szabványos JSON formátumban történnek (express.json() middleware).
- Végpontok (Endpoints): Az API egyértelmű útválasztást (routing) alkalmaz. Külön végpontok felelnek a publikus játékadatok lekéréséért (/game/:id), a hitelesítésért (/api/login), és a védett adminisztrációs műveletekért (pl. POST /api/admin/delete). A védett végpontokon dedikált middleware (requireAdmin) validálja a jogosultságokat.

4.8 Adatbázis Modell

Az adatszerkezet logikai felépítése NoSQL elveket követ.

GAMEHUNT

- Séma definíció: A Mongoose ODM (Object Data Modeling) eszközzel definiált sémák biztosítják a validációt. A felhasználói entitás egy központi kollekcióban kap helyet.
- Tárolt attribútumok: A rekordok tartalmazzák a profiladatokat (username, email), a Bcrypt algoritmussal titkosított jelszavakat, a jogosultsági flageket (isAdmin, adminLevel, isAdult), valamint egy tömböt a kedvenc játékok referenciáival (favorites).

4.9 MongoDB

A fizikai adattárolást egy felhőalapú MongoDB Atlas klaszter végzi.

- Kapcsolatfelépítés: A szerver a MONGO_URI karakterlánc segítségével csatlakozik az adatbázishoz.
- Robusztusság: A hálózati megszakítások és a felhőbeli késleltetések kezelésére a rendszer egy egyedi újrapróbálkozási logikát (retry mechanism) implementál. Sikertelen csatlakozás esetén a szerver meghatározott időközönként (10, 30, majd 60 másodperc) újra megkísérli a kapcsolatfelvételt, garantálva a magas rendelkezésre állást.

4.10 IGDB API

A Gamehunt rendszer elsődleges adatforrása a Twitch által üzemeltetett IGDB (Internet Game Database) API, amely robusztus, valós idejű metaadatokat szolgáltat a játékokról. A szerver és az API közötti kommunikáció dedikált biztonsági és lekérdezési protokollokra épül.

- **Hitelesítési protokoll (OAuth2):** A rendszer egy Server-to-Server hitelesítési modellt alkalmaz. A CLIENT_ID és CLIENT_SECRET környezeti változók felhasználásával a szerver egy ideiglenes access_token-t igényel a Twitch hitelesítési szerverétől.
- **Token menedzsment (Caching):** Az API hívások optimalizálása és a hálózati overhead csökkentése érdekében a backend memóriában gyorsítótárazza a tokent (cachedToken) és annak lejárati idejét (tokenExpiration). Új token lekérésére kizárólag a meglévő lejárata előtt kerül sor.
- **Lekérdezési nyelv (Apicalypse):** Az adatok kinyerése speciális, szöveges formátumú POST kérésekkel (Apicalypse szintaxis) történik. Ez lehetővé teszi a pontos mezőkialakítást (pl. fields name, cover.url, platforms.name), a komplex szűréseket (pl. themes != 42 a felnőtt tartalmak elrejtésére vendég módban), és az eredmények szerveroldali limitálását.

4.11 Node.js Szerver

A backend architektúra magja az express keretrendszerre épülő, eseményvezérelt Node.js webszerver, amely a kliens kérések fogadásáért és az üzleti logika végrehajtásáért felel.

- **Köztesrétegek (Middleware):**
  - **CORS (Cross-Origin Resource Sharing):** Engedélyezi és szabályozza a különböző domainekről érkező hálózati kéréseket.

GAMEHUNT

    - **JSON Body Parser (express.json()):** A beérkező HTTP kérések (payload) JSON formátumú adatainak automatikus feldolgozása és JavaScript objektummá alakítása.
    - **Static File Serving (express.static()):** A public könyvtárban található statikus webes állományok (HTML, CSS, JS, képek) közvetlen és gyors kiszolgálása.
- **Életciklus és Hibatűrés:** A szerver robusztus hibakezeléssel rendelkezik. Indításkor aszinkron módon kapcsolódik a MongoDB klaszterhez. Bár a kód tartalmaz előkészített struktúrákat a kapcsolat újrapróbálkozására (retry logic), az alapértelmezett indítás a mongoose.connect ígéretének (Promise) feloldására épül.
- **Környezeti izoláció:** A szenzitív adatok (MONGO_URI, API kulcsok) a dotenv modul segítségével, a .env fájlból töltődnek be a process.env globális objektumba, elkerülve a hardcoded biztonsági kockázatokat.

4.12 Endpointok

A szerver által exponált RESTful API végpontok három fő logikai kategóriába sorolhatók: Publikus adatkérések, Felhasználói hitelesítés és interakció, valamint Védett adminisztrációs útvonalak.

HTTP Met	Végpont (Path)	Funkció és Működés	Jogosultság

GAMEHUNT

ód us			
GET	/games/popular	Lekéri az aktuálisan legnépszerűbb játékokat az IGDB-ből, figyelembe véve a kliens által küldött isAdult paramétert a tartalomszűréshez.	Publikus
GET	/games/search	Szöveges keresést hajt végre a játékok címében a megadott q (query) paraméter alapján.	Publikus
GET	/game/:id	Részletes metaadatokat (fejlesztők, platformok, megjelenés, képernyőképek) szolgáltat egy adott játékhoz annak egyedi azonosítója (ID) alapján.	Publikus
POST	/api/register	Új felhasználó létrehozása. Validálja az egyediséget, Bcrypt-tel titkosítja a jelszót, és kiszámítja	Publikus

GAMEHUNT

		a nagykorúságot a születési dátumból.	
POST	/api/login	Hitelesíti a felhasználót név és jelszó alapján. Visszaadja a profil alapadatait és a jogosultsági szinteket.	Publikus
POST	/api/favorites/toggle	Hozzáadja vagy eltávolítja a megadott játék ID-t a felhasználó favorites tömbjéből.	Hitelesített
GET	/api/favorites/:username	Visszaadja a felhasználó által kedvelt játékok teljes adatobjektumait az IGDB API-n keresztül.	Hitelesített
GET	/api/admin/users	Kilistázza a rendszerben regisztrált összes felhasználó releváns adatait az adminisztrációs táblázathoz.	**Admin (Lvl 0-3)**
POST	/api/admin/delete	Véglegesen töröl egy felhasználói fiókot az adatbázisból.	**Admin (Szint függő)**

GAMEHUNT

POST	/api/admin/toggle-adult	Átkapcsolja a célfelhasználó isAdult státuszát (csak Level 0 és 1 adminok számára engedélyezett).	**Admin (Lvl 0-1)**
POST	/api/admin/change-role	Módosítja egy felhasználó adminisztrátori szintjét (Level 0-3 között).	**Admin (Lvl 0)**

5. Tervezés és Design

A vizuális tervezés során az elsődleges szempont a szoftveres esztétika és a funkcionális átláthatóság volt. A design támogatja a tartalomközpontú böngészést, minimalizálva a szükségtelen vizuális zajt.

5.1 UI/UX Tervezés

Az alkalmazás felhasználói felülete a Szemantikus UI elveit követi, ahol az elemek elrendezése tükrözi azok fontossági sorrendjét.

- Layered Design: A felület rétegelt felépítésű; a háttérben egy dinamikus rácsháló (bg-grid-layer)

GAMEHUNT

biztosít mélységet, amely felett a kártyák és panelek lebegő hatást keltenek.
- Interakciós modellek: A rendszer elkerüli a hagyományos oldallekéréseket. Minden fontos művelet (bejelentkezés, játék részletek, adminisztráció) modális ablakokban vagy oldalsó panelekben (drawers) történik, megőrizve a felhasználó kontextusát.
- Vizuális visszacsatolás: Az interaktív elemek (gombok, kártyák) hover-effektusokkal és finom átmenetekkel (transition: 0.3s) reagálnak a felhasználói beavatkozásra.

5.2 Színpaletta

A projekt egyik kiemelt technikai megoldása a natív CSS változókon (variables) alapuló kettős témakezelés. A theme.js aszinkron módon kezeli a váltást, az állapotot pedig a localStorage tárolja a munkamenetek közötti konzisztencia érdekében.

Elem megnevezése	Sötét téma (Dark)	Világos téma (Light)
Alapértelmezett háttér	#121212	#f0f2f5
Kártyák és panelek	#1e1e1e	#ffffff

GAMEHUNT

Elsődleges szövegszín	#ffffff	#000000
Oldalsáv háttér	#0a0a0a	#e2e5ea
Kiemelő szín (Accent)	#ffffff50 (áttetsző)	#bdbdbd50 (áttetsző)

5.3 Reszponzivitás

Az alkalmazás "Adaptive Layout" megközelítést alkalmaz, biztosítva a hibátlan megjelenést asztali környezettől a mobil eszközökig.

- Rugalmas rácsrendszer: A játéklistázás CSS Grid és Flexbox technológiát használ, amely automatikusan újratördeli a kártyákat a rendelkezésre álló szélesség alapján.
- Media Queries: A stíluslap dedikált szabályokat tartalmaz a 768px alatti szélességű kijelzőkhöz:
    - Mobil navigáció: Az oldalsáv (sidebar) pozíciója és elrendezése megváltozik, hogy érintéssel is könnyen kezelhető legyen.
    - Kártya optimalizáció: Mobilon a játék-kártyák vertikális elrendezésre váltanak, a képek és szövegek középre igazítva jelennek meg a maximális olvashatóság érdekében.
    - Interaktív elemek: A modális ablakok és a keresőmező szélessége mobilon kitölti a

GAMEHUNT

képernyőt (100%), elkerülve az horizontális görgetést.

6. Tesztelés

A Gamehunt alkalmazás fejlesztése során kiemelt figyelmet fordítottunk a kódminőségre és a funkcionális stabilitásra. A tesztelési stratégia alapját az automatizált egység- és integrációs tesztek képezik.

6.1 Jest Tesztek

A backend logika verifikációjához a **Jest** tesztkeretrendszert és a **Supertest** könyvtárat alkalmaztuk. Ez a párosítás lehetővé teszi a HTTP végpontok szimulációját és a válaszok validálását anélkül, hogy a szervert manuálisan el kellene indítani.

Tesztelési környezet és i zoláció:

- **Mocking technológia:** A tesztek során "mock" (utánzat) objektumokat használunk a külső függőségek izolálására. Ez biztosítja, hogy a tesztek gyorsak legyenek és ne függjenek külső tényezőktől:
    - **Adatbázis:** A `mongoose` kapcsolatot és a modelleket (pl. `User.findOne`) Spy-objektumokkal helyettesítjük, így nem szükséges valós MongoDB futtatása.

    - **Külső API:** Az IGDB (Twitch) API hívásokat a `node-fetch` modul mockolásával szimuláljuk, fix választ adva a lekérdezésekre.
- **Környezeti konfiguráció:** A tesztek futtatásához egy dedikált `test` környezetet definiáltunk, elkerülve a fejlesztői adatok véletlen módosítását.

Lefedett funkcionális területek:

1. **Hitelesítési folyamatok (Authentication):**
    - **Regisztráció:** Új felhasználó létrehozásának ellenőrzése (201 Created) és a hiányzó adatok (400 Bad Request) kezelése.
    - **Bejelentkezés:** Sikeres azonosítás és a hibás hitelesítési adatok (pl. nem létező felhasználó) validálása.
2. **API Integráció:**
    - **Játéklekérés:** Annak igazolása, hogy a `/games/popular` végpont megfelelően továbbítja-e a kéréseket az IGDB felé, és helyesen adja-e vissza a formázott JSON választ.
3. **Adminisztrációs műveletek:**
    - A tesztek szimulálják a különböző jogosultsági szinteket.
    - **Jogosultság-kezelés:** Annak ellenőrzése, hogy az admin végpontok (törlés, átnevezés, életkor-státusz módosítás) csak megfelelő `adminLevel` esetén engedélyezik a műveletet.
    - **Adatbiztonság:** Verifikáljuk, hogy egy adminisztrátor nem törölheti saját magát, illetve magasabb szintű admint sem.
4. **Felhasználói interakciók:**

GAMEHUNT

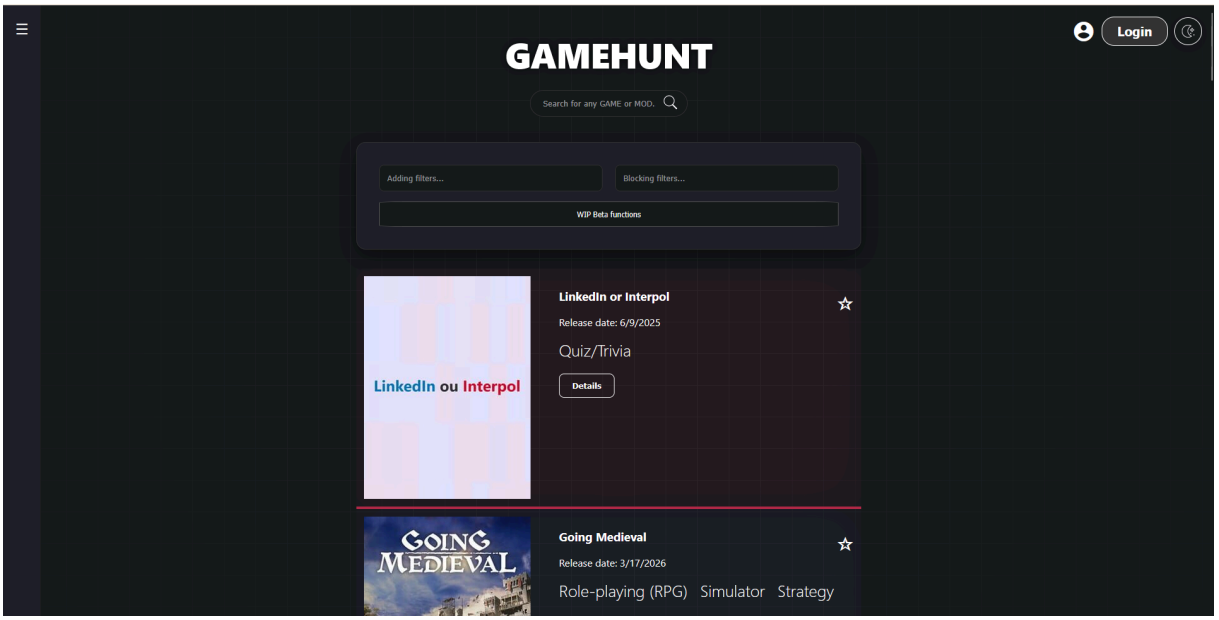
- **Favorites Toggle:** A kedvencek listájának módosítását végző logika tesztelése, biztosítva az aszinkron adatbázis-módosítások sikerességét.

Futtatás: A tesztek az `npm test` parancs kiadásával futtathatók, amely részletes jelentést ad a sikeres és sikertelen esetekről, segítve a regressziós hibák azonnali felismerését.

7. Felhasználói Dokumentáció

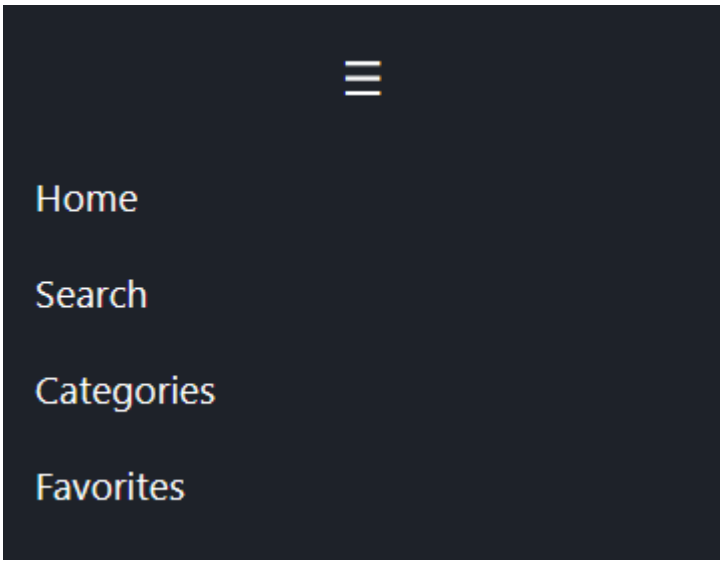
7.1 Navigáció a Kezdő Oldalon

Az oldalt betöltve

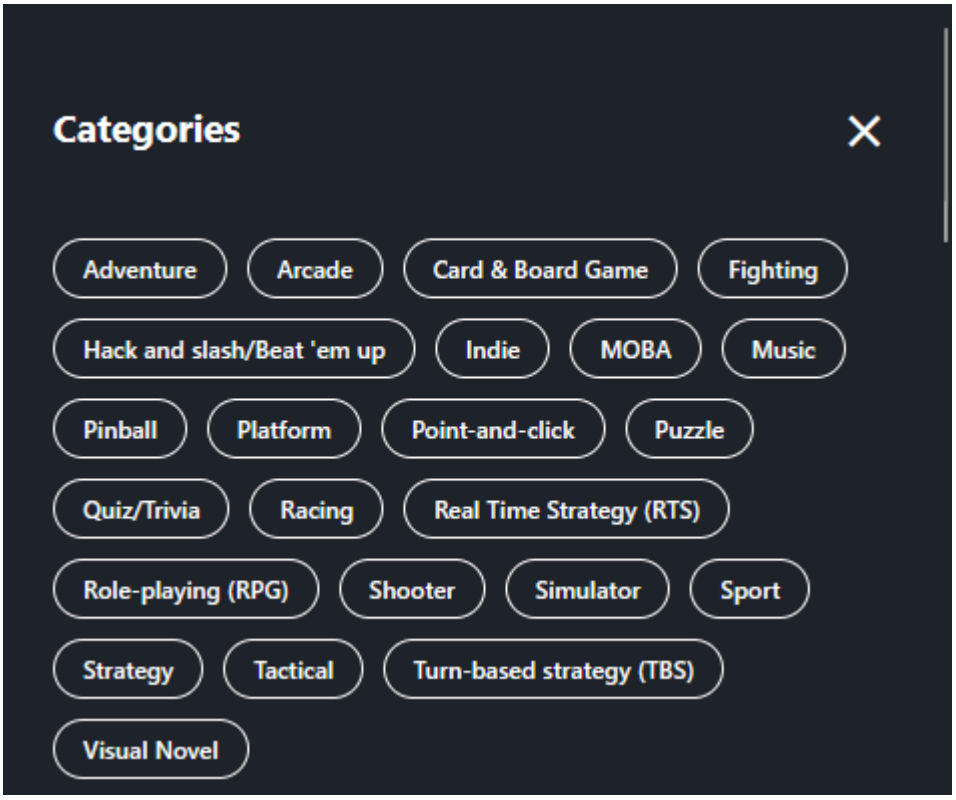


szembetűnik a projekt címe középen felül, alatta látható a keresőmező, alapból a felhasználó lát betöltött játékokat, ezek a felhasználó ízvilágától függetlenül jelennek meg az elmúlt hétben a legfelkapottabbak közül.

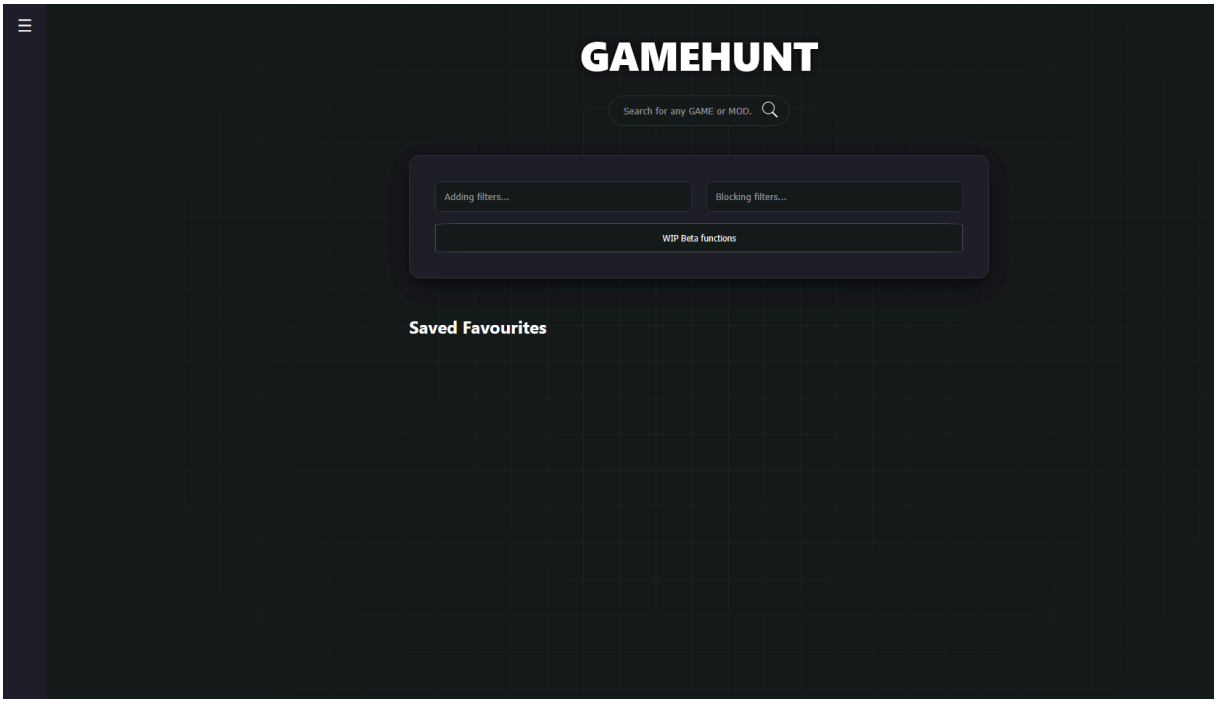
Desktop nézetben bal oldalon látható egy fül



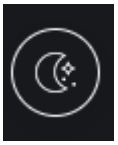
amin a felhasználó lát egy “Home” gombot ami újratölti az oldalt, egy “Search” gombot ami a keresőre viszi a felhasználót, egy “Categories” gombot ami jobb oldalról megnyit egy fület



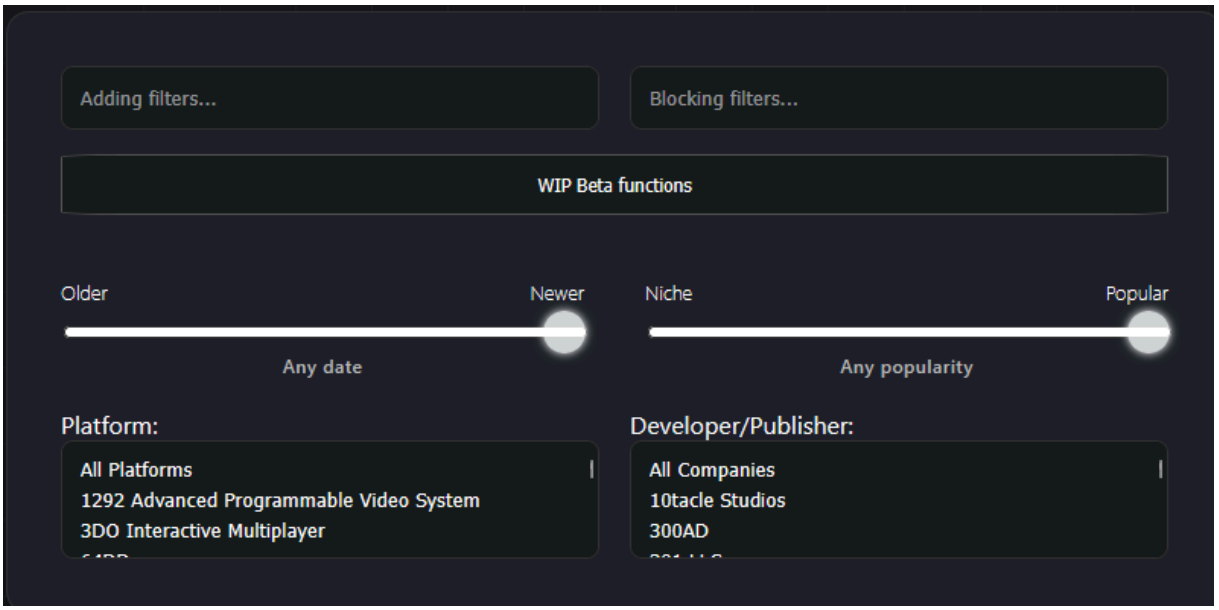
amin kategóriákból böngészhet a felhasználó, ezt a jobb felső sarokban egy “x”-szel bezárhatja, és a “Categories” gomb alatt egy “Favorites” Gomb ami a (bejelentkezett) felhasználó saját kedvencek könyvtára



amihez a kártyákon fellelhető csillaggal tud ide behelyezni. Jobb oldalt a felhasználó lát fent a sarokban

egy Téma változtató  (DARK/LIGHT) gombot , mellette attól függően hogy belépett e fiókjába:
-ha nem akkor “Login”
-ha igen akkor “(Felhasználónév)”
És ha bejelentkezve van a felhasználó akkor ki tud lépni a fiókból.
még középen a kereső alatt egy kategória hozzá- és elvévő és egy későbbi fejlesztésre használatos eltüntető és megjelenítő gomb amivel előtűnnek fejlesztés alatti

kereső funkciók.



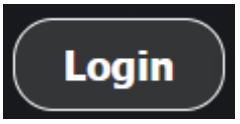
7.2 Játékok szűrése

A keresőmező amin keresztül a felhasználó elküldi a keresett játék címét, azt az oldal végigvezeti az API-n és mikor megérkezett a futtatott szerver felé, majd visszaküldi a találatokat (max 500db / keresés) a felhasználónak.

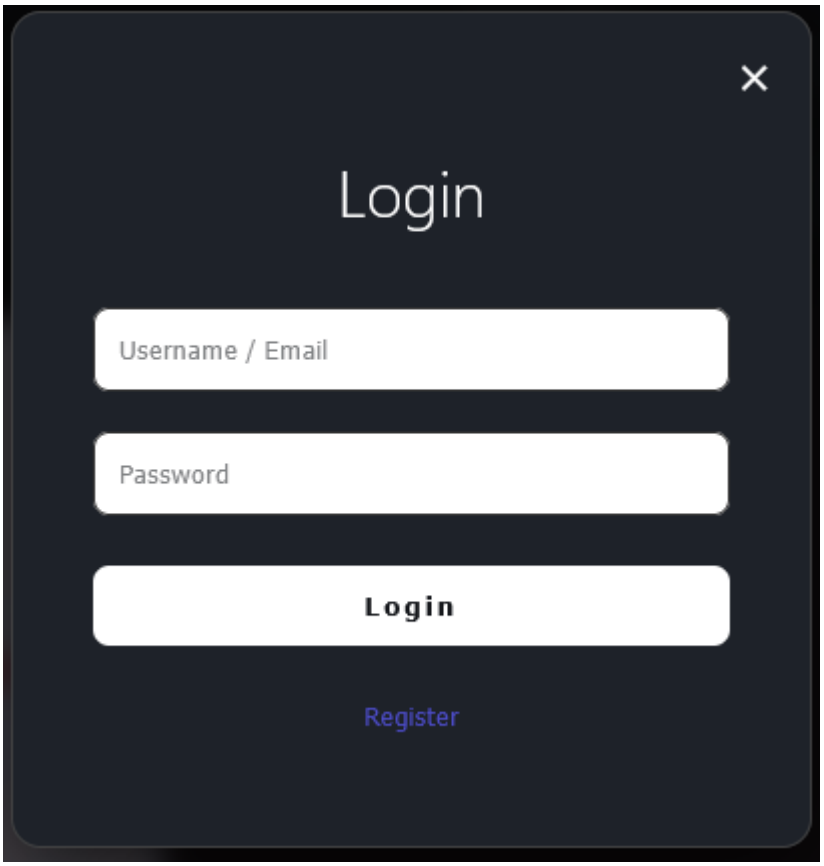
Ezeket a találatokat a felhasználó lefelé haladva böngészheti, majd a legaljára érkezve ha elérte az oldal maximális 20 játékot (/oldal) amennyiben több játék van, egy léptetővel tud tovább menni esetleg ha nem találta meg a neki megfelelő játékot vagy csak böngészni szeretne, tud menni tovább egészen 500 játékig.

A pontosabb szűrésre még használatos lehet kategóriákkal kapcsolatos “hozzá/elvétel”. Alatta a béta funkciók megjelenítése gombbal előtűnik még “slider” ami a dátumot és a játék hírességét veszi alapul alatta pedig egy Platform és vagy Dev/Publisher kiválasztó. Fontos megjegyezni hogy ezek “béta” funkciók.

7.3 Bejelentkezés

Az előbb említett “login” gombra 

kattintva megjelenik egy ablak

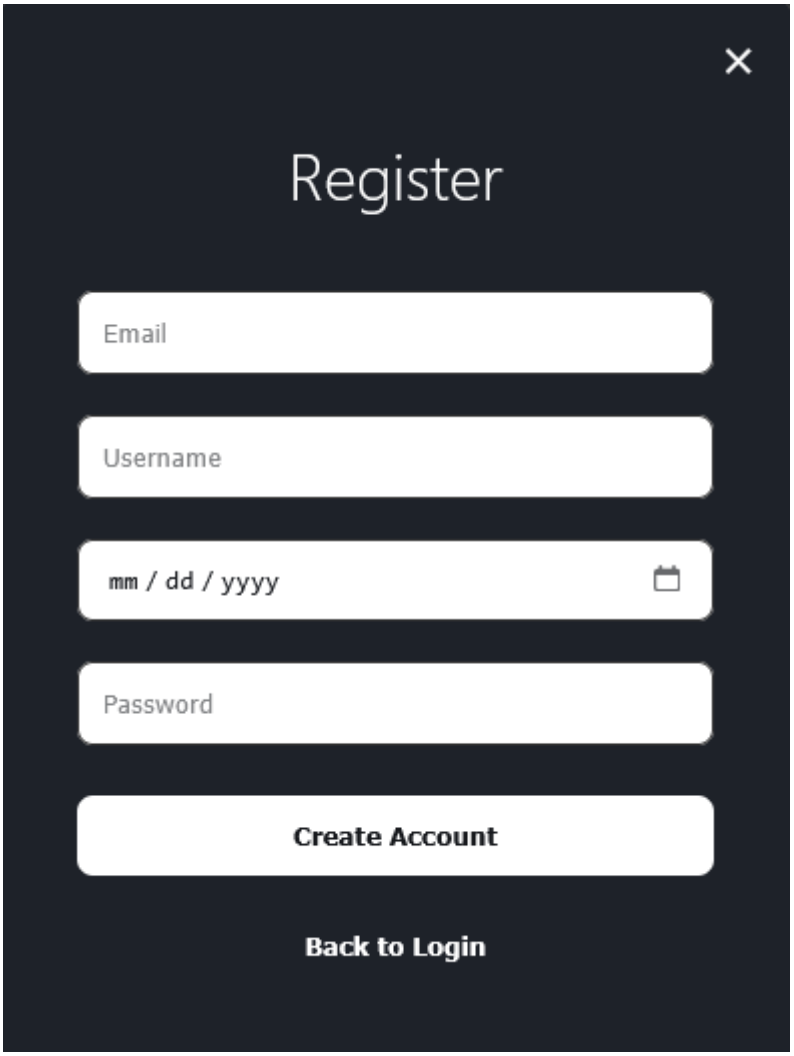


amiben a felhasználó, amennyiben regisztrált a regisztrált adatokat a megfelelő helyre beírva (username/email,password) majd “login”-ra nyomva bejelentkezik és az előbb még login mostmár a felhasználó neve lesz jobb felső sarokban, és a felhasználó hozzáfér a githubon feltüntetett előnyökkel, mint például a “favorites” oldal és a játékok beküldése oda.

7.4 Regisztrálás

A felhasználó amennyiben nem regisztrált még, a “login” gomb alatt kéken megtalálható register szövegre Register kattintva megtalálja a regisztrációs fület ahol megadja e-mail címét felhasználó nevét, születési dátumát és jelszavát.

GAMEHUNT



(Jelszó Emlékeztető vagy Adatbázison belüli jelszóváltás, fiók törlése még nincs.).

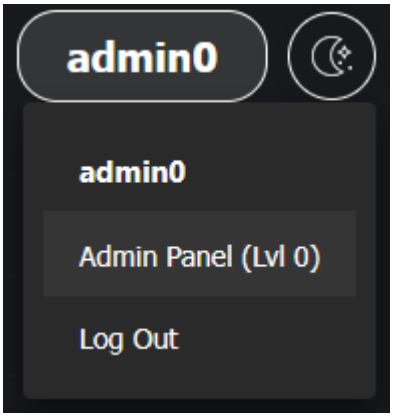
7.5 Kedvencek Hozzáadása

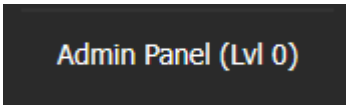
A felhasználó a feltüntetett és kiválasztott játék kártyájának jobb felső sarkában látható csillag ikon kattintásával hozzáadja a kedvenceihez (be”login”-olva) és a baloldalon “Favourites”-re kattintva a kedvencek oldalán azt esetlegesen megnézheti vagy kiszedheti onnan.

7.6 Admin Felület

GAMEHUNT

Az oldalon 2 féle képpen lehet hozzáférni az admin felülethez, ha az adatbázisból manuálisan adják hozzá az isAdmin-t vagy ha egy másik adminisztrátor adja hozzá amit az admin felületen meg tud tenni.

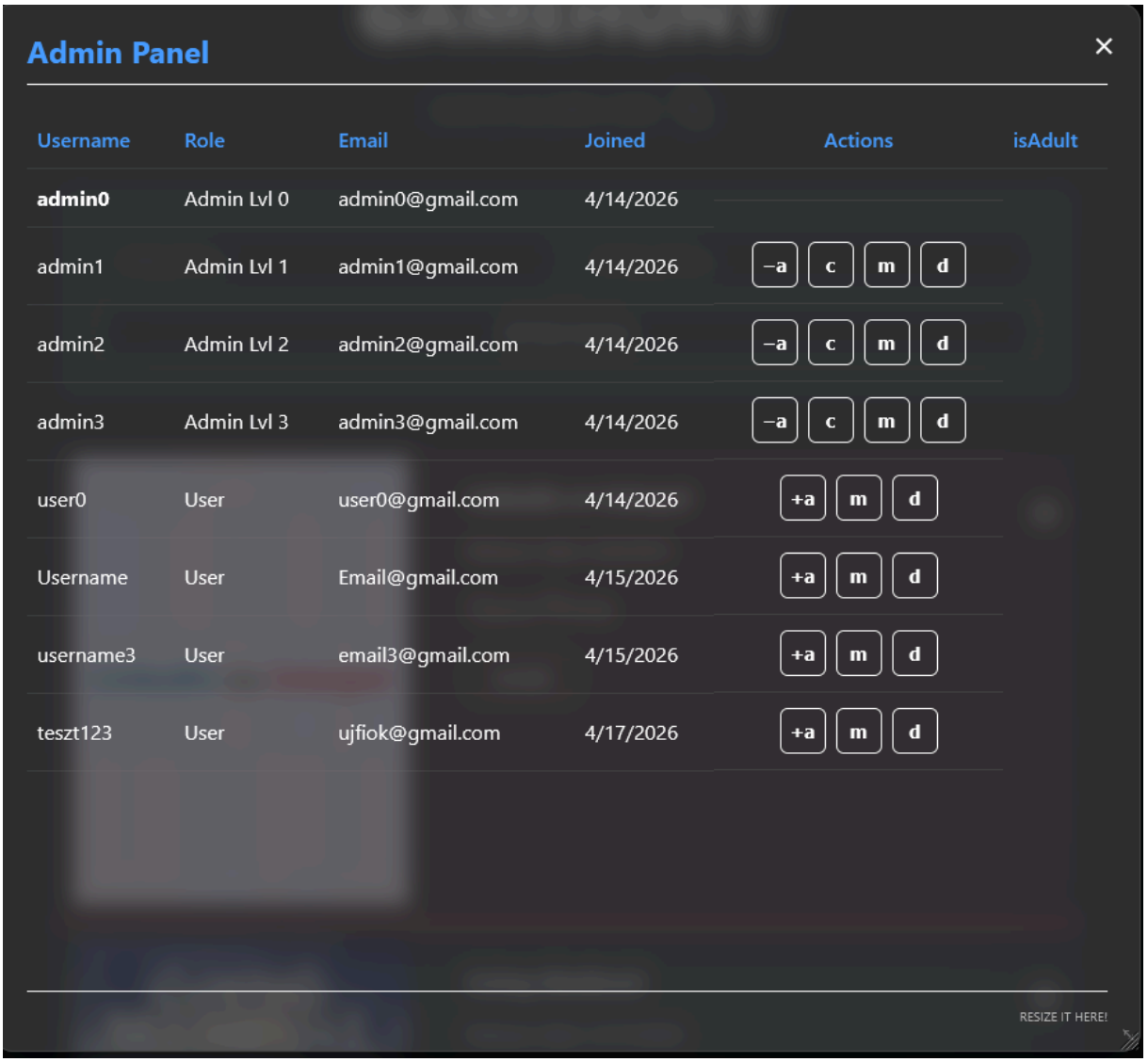




Maga az admin felület a felhasználónévre kattintva jobb felül sarokban majd az adminpanel-re kattintva egy olyan felület jön fel ahol az adatbázis összes felhasználója, nekik az emailcímük,regisztráció dátuma, role-ja (admin/user), admini szinttől függően tud (4gomb) 1. hozzáadni vagy törölni admin-t, 2. admin szinit változtatása gomb 3. felhasználónév változtatása gomb 4. felhasználó teljes törlése az adatbázisból,

GAMEHUNT

és még az isAdult ami később lesz elérhető.



Ez az admin “modal” állítható nagyságú (lásd jobb alsó sarok), a username-re és vagy az email-re kattintva az admin egy vagy több adott felhasználó keresése könnyített.

8. Továbbfejlesztési Lehetőségek

8.1 Rövid Távú

Béta funkciók teljes kidolgozása, Trelló befejezése

GAMEHUNT

8.2 Középtávú

További adatok bekérése, több adat megjelenítése az oldalon

8.3 Hosszú távú

Kereső “tippelése” mire akar keresni a felhasználó. És adatok felhasználása a felhasználó élményének, betöltött játékok érdekében

9. Linkek és Források

A projekt transzparenciája és reprodukálhatósága érdekében az alábbiakban rögzítjük a forráskód helyét, a felhasznált külső modulokat és a projektmenedzsment felületet.

9.1 GitHub Repository

A projekt teljes forráskódja, verziótörténete és az egyes modulok (frontend, backend) implementációja az alábbi távoli tárhelyen érhető el:

- Elérhetőség: https://github.com/23BAttila/Vizsgaremek/

9.2 Használt NPM Csomagok

GAMEHUNT

A rendszer backend rétege Node.js környezetben készült. Az alábbi táblázat a futtatáshoz elengedhetetlen külső függőségeket (dependencies) és azok verziószámait tartalmazza a package.json fájl alapján:

Csomag név	Verzió	Funkció leírása
bcryptjs	^3.0.3	Felhasználói jelszavak biztonságos, sózott hashelése.
cors	^2.8.6	Cross-Origin Resource Sharing (erőforrás-megosztás) kezelése.
dotenv	^17.4.1	Környezeti változók (.env) konfigurációja.
express	^5.2.1	Minimalista webes keretrendszer az API kiszolgáláshoz.
mongoose	^9.4.1	MongoDB objektummodellező könyvtár (ODM).
node-fetch	^2.7.0	HTTP kérések kezelése (pl. külső IGDB API hívásokhoz).

GAMEHUNT

uuid	^13.0.0	Egyedi, univerzális azonosítók (UUID) generálása.

9.3 Munkamenet Nyomonkövetése

A fejlesztési szakaszok koordinálása, a feladatok (backlog, in progress, done) adminisztrációja és a csapattagok közötti kollaboráció az alábbi felületen történt:

- Projektmenedzsment (Trello): https://trello.com/b/yHRLy9VS/jatek-ajanlo

10. Összefoglalás

10.1 A Projekt Státusza

A projekt prezentálható állapotban

10.2 Elért Célok

Tudás bővítése

10.3 Tanulságok

GAMEHUNT

Időbeosztás kezelése, munkák megosztása

10.4 Elérhetőségek

Email: bojtosatika2006@gmail.com
Iskola által szolgáltatott: 21.bojtos.attila@mechwart.com

GAMEHUNT